SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1715**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 20%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I Kurangi Arun Kumar/192372287 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

Answer :

Greedy Best-First Search (GBFS)

- **Behavior:** GBFS expands nodes strictly based on the heuristic evaluation function, $h(n)$, which estimates the remaining distance from node n to the destination (straight-line distance). It completely ignores path cost accumulated so far, $g(n)$.
- **Strengths:**
  - Extremely fast path generation under ideal conditions.
  - Low computational overhead, allowing rapid decision-making under high-urgency constraints.
- **Weaknesses under Uncertainty:**
  - Highly susceptible to local optima, dead ends, and blocked paths caused by floodwaters.
  - Ignores variable costs like weather headwinds or high-risk terrain, often selecting visually direct paths that end up taking significantly longer or putting the drone at risk.
  - Incomplete and non-optimal in dynamic, real-world environments.

*A Search Algorithm**

- **Behavior:** A* selects paths using the total estimated cost function $f(n) = g(n) + h(n)$, balancing actual traveled cost $g(n)$ with the estimated remaining distance $h(n)$.
- **Cost vs. Heuristic Balance:**
  - **$g(n)$ (Cost so far):** Incorporates real-time movement costs, terrain penalties, and severe weather delays.

    - **$h(n)$ (Heuristic):** Directs search progress toward the target location to minimize unnecessary exploration.

- **Handling Dynamic Conditions:** By keeping track of $g(n)$, A* actively avoids paths that accumulate excessive risk or delay, switching to safer, lower-cost alternative routes even if they temporarily diverge from the straight-line heading.

Impact of Heuristic Accuracy

- **Admissible & Consistent Heuristics:** Straight-line distance is admissible (never overestimates real flight distance) and consistent (satisfies the triangle inequality). This guarantees that A* will find the optimal path.

- **Impact on GBFS:** Accurate heuristics keep GBFS on a direct trajectory toward the goal. However, if a straight-line route is blocked by sudden flooding or high winds, GBFS becomes erratic—backtracking inefficiently or getting trapped in long, suboptimal loops.

- *Impact on A:**

    - **Underestimation (Inaccurate/Too Low):** Forces A* to expand more state space nodes, increasing computation time but retaining path optimality.

    - **Overestimation (Inadmissible):** Makes A* behave more like GBFS, losing its guarantee of optimality to save time.

Dynamic Changes and Blocked Paths

- **Re-Planning Overhead:** When a path is suddenly blocked, both standard static algorithms must recalculate routes from the drone's current position.

- **GBFS Impact:** Can repeatedly commit to dead-end paths near obstacles because $h(n)$ continues pointing directly toward the target across the blocked zone.

- *A Impact:** Automatically reweights paths once the blocked node cost jumps to infinity. However, repeated global re-planning on complex maps introduces computational delays during emergency flight.

Algorithm Recommendation & Justification

- **Recommendation:** *A Search** (or an incremental variant like *D Lite** for real-time edge updates).

Metric	Greedy Best-First Search	A* Search
Optimality	No	Yes (Guaranteed)
Safety & Terrain Awareness	Poor (Ignores $g(n)$)	High (Factors in weather/terrain risk)
Real-Time Responsiveness	Fast	Moderate to High

Justification: While GBFS offers quick path decisions, its lack of path-cost awareness makes it dangerously unsafe for emergency medical deliveries where terrain risk, weather delays, and dynamic flood barriers exist. A* guarantees the optimal balance between flight safety, actual travel cost, and destination arrival time.

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

Answer :

- **Optimization Formulation:**
    - **State Space (S):** The complete set of signal timing configurations across all intersections (e.g., green-light durations, phase sequencing, offset timing between adjacent lights).
    - **Objective Function ($f(s)$):** Minimize total waiting time, fuel consumption, and vehicle queue lengths.

$$\min_{s \in S} f(s) = w_1 \cdot \text{Wait Time}(s) + w_2 \cdot \text{Fuel Consumption}(s) + w_3 \cdot \text{Congestion Index}(s)$$

    - **Constraints:** Minimum/maximum green phase bounds, pedestrian crosswalk clearance intervals, and emergency vehicle priority rules.

- **Traditional Search Inefficiency:** Uninformed (BFS, DFS) and informed (A*, Dijkstra) search algorithms fail because:
    - **Combinatorial Explosion:** Hundreds of interconnected signals create a high-dimensional continuous search space impossible to explore exhaustively.
    - **Dynamic State Shifts:** Traffic conditions drift continuously; fixed path-finding search trees cannot adapt to fluctuating vehicle flows without full, costly re-computations.

Hill Climbing Mechanics & Limitations

- **How It Works:** Starts with a baseline signal timing layout. Evaluates neighboring timing adjustments (e.g., adding 5 seconds of green light to a heavily congested main road) and moves exclusively to neighbors that decrease the objective function $f(s)$.
- **Core Limitations:**
    - Operates strictly greedily with no memory or global perspective.
    - Cannot accept temporarily worse traffic metrics to escape suboptimal local solutions.

Search Landscape Obstacles (Traffic Interpretation)

- **Local Maxima (Suboptimal Minima):** A localized signal balance where tweaking any single signal timing worsens nearby traffic, even though a radically different synchronization pattern across the entire district would clear overall congestion.

- **Plateaus:** Extended regions in the state space where modifying signal lengths by small increments produces near-zero changes in vehicle wait times. The algorithm wanders aimlessly without direction.

- **Ridges:** Narrow paths of continuous improvement (e.g., precisely linked green waves along a primary corridor). Standard single-variable modifications fail to stay on the ridge, repeatedly bouncing back and forth across suboptimal states.

Overcoming Limitations: Advanced Metaheuristics

- **Simulated Annealing (SA):**
    - **Mechanism:** Accepts suboptimal timing tweaks with a probability determined by a temperature parameter T, where $P(\text{accept}) = \exp(-\Delta E / T)$.
    - **Traffic Impact:** High initial temperature allows the system to drastically alter city-wide signal coordination during early phases, escaping local congestion traps before gradually stabilizing into fine-tuned adjustments as T drops.
- **Genetic Algorithms (GA):**
    - **Mechanism:** Maintains a population of signal configurations. Combines successful timing traits (Crossover) and introduces random timing modifications (Mutation) across generations.
    - **Traffic Impact:** Explores multiple diverse traffic strategies in parallel, making it highly effective at discovering non-obvious corridor-wide coordination patterns.

Recommendation & Justification

- **Recommendation: Simulated Annealing** for localized, high-speed adaptive signal control—or a **Hybrid Genetic-SA / Deep Reinforcement Learning** architecture for large-scale city deployment.

Metric	Hill Climbing	Simulated Annealing	Genetic Algorithms
Scalability	High (Low Compute)	Moderate to High	Low to Moderate (High Overhead)
Adaptability	Very Poor	High	High
Escapes Local Optima	No	Yes	Yes

Justification: While Genetic Algorithms are robust at finding global optima, their high computational cost makes real-time citywide adaptation challenging. Simulated Annealing provides the ideal balance between low latency and the ability to escape local traffic bottlenecks, making it highly suitable for real-time traffic signal optimization.

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with

limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

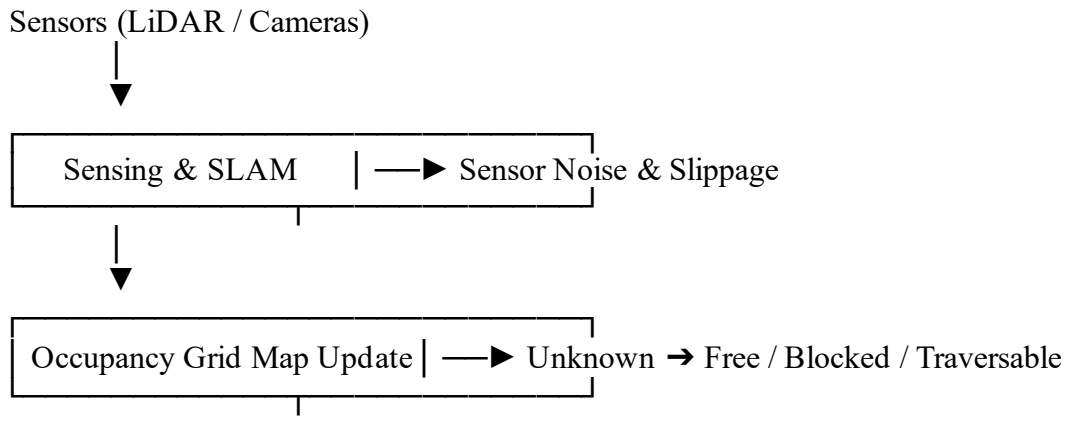
Answer :
Online Search vs. Offline Search

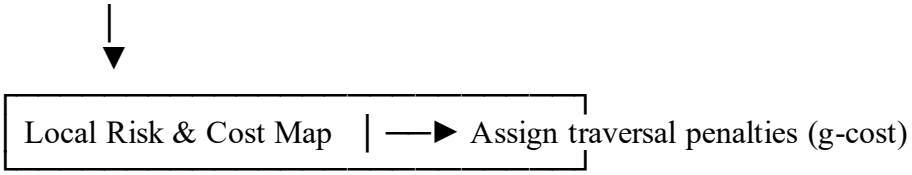
- **Why Online Search Is Required:**
    - **Offline Search Assumption:** Requires a complete, static world model to calculate an end-to-end path prior to execution (e.g., standard A* or Dijkstra on a fully known map).
    - **Mars Environment Reality:** The terrain is unknown, partially observable, and dynamic. An offline planner would generate a path based on missing or coarse satellite data, causing immediate collision or entrapment upon encountering unexpected obstacles.
    - **Execution Interleaving:** The rover must alternate continuously between **sensing**, **planning**, **acting**, and **learning** (updating its local map) in real time.

Challenges of Partial Observability & Dynamic Environments

- **Partial Observability:** Sensors (LiDAR, stereo cameras) have limited range, blind spots, and range noise. Terrain beyond the visual horizon remains completely hidden until physically approached.
- **Dynamic & Uncertain Hazards:** Loose sand traps (slippage), shifting wind-blown dust, and unstable rocks alter travel risk and locomotion costs unpredictably.
- **Irreversible Actions:** Physical mistakes on Mars (e.g., slipping into a steep crater) are permanent and catastrophic. Online search must prioritize safe exploration over aggressive pathfinding.

Building & Updating the Internal Model





1. **Local Occupancy Grid:** Represents the environment as a probabilistic 2D or 3D grid, categorizing cells as *Free*, *Blocked*, or *Unexplored*.

2. **SLAM Integration:** Uses Simultaneous Localization and Mapping (SLAM) to estimate rover position relative to newly observed landmarks while filtering out sensor noise.

3. **Cost Mapping:** Translates terrain attributes (slope angle, surface roughness) into continuous traversal costs $g(n)$, continuously refining the internal belief state as new sensors scan the immediate surroundings.

Comparison of Search Strategies

- **Uninformed Search (BFS, DFS):** Exhaustive and state-agnostic. Entirely unsuited for physical robotics due to high state expansion costs, long computational delays, and lack of safety awareness.

- *Informed Static Search (Standard A):** Highly efficient with an accurate heuristic, but degrades rapidly under partial observability—requiring a full global recalculation from scratch every time an unexpected hazard blocks a path.

- *Online Local Search (LRTA, D* Lite, PHA*):

    - Computes immediate next actions using local sensor horizon data and estimated remaining cost to goal $h(n)$.

    - Updates heuristic values dynamically based on observed dead ends to prevent getting trapped in infinite loops.

Method Justification & Recommendation

- **Recommendation:** *Incremental Replanning Search (D Lite / Field D*) integrated with Risk-Aware Online Search (LRTA*).

Justification: D* Lite searches backward from the destination goal to the rover's current position. When local sensors detect new terrain hazards or blocked paths, it incrementally updates only the affected nodes in its search tree rather than recalculating the entire global route. This delivers the real-time execution speeds required for planetary rovers while maintaining safety and low computational overhead.

Metric	**Offline Search (A*)**	**Standard Online Search (LRTA*)**	**Incremental Replanning (D* Lite)**
Complete Map	Yes	No	No

Required?			
Re-planning Speed	Slow (Full recalculation)	Fast (Local adjustments)	Very Fast (Reuses previous search trees)
Safety & Optimality	Fails under uncertainty	Suboptimal, complete	Near-optimal, highly adaptable

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

Answer :
CSP Problem Formulation

- **Variables (V):** The set of courses/exams that must be scheduled.

$$V = \{C_1, C_2, C_3, \dots, C_n\}$$
(Optionally paired with student accommodation flags if handled as distinct sub-events).

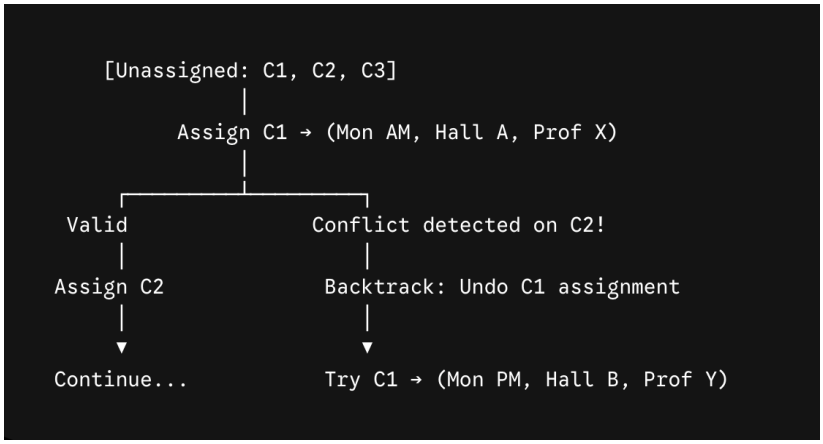
- **Domains (D):** For each variable C_i, the domain D_i represents all valid combinations of time slots, exam halls, and assigned invigilating faculty.

$$D_i = \{ (t, h, f) \mid t \in T, h \in H, f \in F \}$$
Where T is the set of available time slots, H is the set of physical halls, and F is the set of qualified faculty members.

- **Constraints (C):**
    - **Hard Constraints (Mandatory):**
        - **Student Overlap:** $\text{Slot}(C_i) \neq \text{Slot}(C_j)$ if student enrollment sets $S_i \cap S_j \neq \emptyset$.
        - **Hall Capacity:** Total enrolled students in a slot for hall h cannot exceed $\text{Capacity}(h)$.
        - **Prerequisites/Prerequisites Order:** $\text{Slot}(C_i) < \text{Slot}(C_j)$ for sequential subjects.
        - **Faculty Conflicts:** Faculty f cannot be assigned to two exams in the same slot t.
    - **Soft Constraints (Optimization Preferences):**
        - Minimize student workload (avoid scheduling 3 exams on consecutive days for a single student).
        - Even distribution of exam invigilation load across available faculty.

Backtracking Search Mechanics

Backtracking incrementally builds a complete timetable by assigning values (t, h, f) to variables C_i one at a time.



1. **Selection:** Pick an unassigned course C_i.
2. **Assignment:** Assign a slot/hall/faculty triple from D_i.
3. **Consistency Check:** Evaluate all hard constraints against previously assigned courses.
4. **Backtrack Step:** If a constraint is violated, undo the assignment and try the next value in D_i. If D_i is exhausted, backtrack to the previous course C_{i-1}.

Efficiency via Constraint Propagation & Heuristics

Standard backtracking fails on large universities due to massive search space depth. Integrating heuristics and propagation prunes invalid states early:

- **Forward Checking:** Whenever course C_i is assigned a slot t, immediately scan all unassigned courses C_j that share enrolled students and remove t from their domains D_j. If any D_j becomes empty, backtrack immediately without wasting time searching deeper.
- **Arc Consistency (AC-3 Algorithm):** Maintains consistency across pairs of variables. If assigning a value to C_i leaves no valid options for C_j under constraint $C_{i,j}$, the invalid values are pruned across the network prior to search.
- **Variable Ordering Heuristics:**
    - **MRV (Minimum Remaining Values):** Pick the course with the fewest legal slot choices left ("most constrained variable").
    - **Degree Heuristics:** Pick the course involved in the highest number of student overlap constraints ("most constraining variable").

Real-World Challenges & Scalability Strategy

- **Real-World Bottlenecks:**
    - **Dynamic Changes:** Mid-semester faculty illness or hall maintenance requires fast, local timetable repair without completely rebuilding the entire schedule.
    - **Large Combinatorial Scale:** High student enrollment cross-registering across departments turns CSP into an NP-hard problem.

Strategy	Backtracking alone	CSP + Constraint Propagation	Hybrid CSP + Local Search
Scalability	Extremely Poor	Moderate	High
Handling Soft Constraints	Poor	Average	Excellent
Re-scheduling Speed	Slow	Moderate	Fast

Recommended Strategy: Hybrid Approach (CSP Propagation + Metaheuristic Optimization)

Use CSP with AC-3 and MRV to rapidly generate an initial valid timetable that satisfies all **hard constraints**. Then, apply **Min-Conflicts Local Search** or **Genetic Algorithms** to optimize **soft constraints** (e.g., student exam spacing, faculty balance) and handle real-time modifications efficiently.

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

Answer :

Minimax Problem Modeling

- **Adversarial Framework:** The RTS game is modeled as a two-player zero-sum game where the AI (**MAX**) aims to maximize its advantage, and the human opponent (**MIN**) works to minimize it.

- **Tree Structure:** The search space is structured as an alternating tree of game states:

$$\text{Value}(s) = \begin{cases} \text{Utility}(s) & \text{if Terminal}(s) \\ \max_{a \in A(s)} \text{Value}(\text{Result}(s, a)) & \text{if } \text{Player}(s) = \text{MAX} \\ \min_{a \in A(s)} \text{Value}(\text{Result}(s, a)) & \text{if } \text{Player}(s) = \text{MIN} \end{cases}$$

- **Core Assumption:** Assumes the opponent always plays optimally to counter the AI's moves.

Computational Challenges of Large Game Trees

- **Combinatorial Explosion:** The search tree grows exponentially at $O(b^d)$, where b is the branching factor and d is the depth.

- **RTS Search Bottleneck:** While turn-based games like Chess have a branching factor of $b \approx 35$, RTS games involve simultaneous actions, continuous space, and

hundreds of individual units, raising b to well over 1000.

- **Time Constraints:** Plain Minimax takes minutes or hours per move at meaningful depths, making it impossible to meet real-time frame budgets (e.g., 50–100ms per tick).

Efficiency via Alpha-Beta Pruning

Alpha-Beta pruning removes subtrees that cannot possibly influence the final decision at the root state, skipping unnecessary calculations without sacrificing accuracy.

- **Tracking Parameters:**
    - **α (Alpha):** Highest score **MAX** is guaranteed along the current path (initialized to $-\infty$).
    - **β (Beta):** Lowest score **MIN** is guaranteed along the current path (initialized to $+\infty$).
- **Pruning Rules:**
    - **Beta Cutoff:** At a MAX node, if $\alpha \ge \beta$, search stops because MIN higher up in the tree will never allow this branch.
    - **Alpha Cutoff:** At a MIN node, if $\beta \le \alpha$, search stops because MAX higher up already has a strictly better move.
- **Performance Gain:** Under optimal move ordering (evaluating the strongest moves first), the search space drops from $O(b^d)$ to $O(b^{d/2})$, doubling the search depth reachable in the same time limit.

Evaluation Function Design

Because search must be truncated early before reaching win/loss terminal states, a heuristic function $V(s)$ estimates intermediate board strength:

$$V(s) = w_1 \cdot \text{MilitaryStrength}(s) + w_2 \cdot \text{Economy}(s) + w_3 \cdot \text{MapControl}(s) + w_4 \cdot \text{TechLevel}(s)$$

- **Factor Justification:**
    - **Military Strength (w_1):** Tracks unit counts, remaining health, and tactical unit compositions to gauge combat viability.
    - **Economy & Resources (w_2):** Quantifies worker count, resource collection rate, and unspent capital to support sustained production.
    - **Map Control & Positioning (w_3):** Measures vision, ownership of choke points, high-ground placement, and resource node expansions.
    - **Technology Level (w_4):** Measures unlocked tech tier and unit upgrades, projecting future combat power.

Balancing Optimality and Real-Time Performance

Technique	Function in Real-Time Decision Making
Iterative	Searches depth $d=1, 2, 3 \dots$ sequentially; returns the best move found when

Deepening	the frame budget expires.
Transposition Tables	Uses hash tables to cache evaluated game states and eliminate redundant calculations.
Move Ordering	Evaluates high-value actions (captures, unit kills) first to maximize early Alpha-Beta cutoffs.

Recommended Strategy: Use **Iterative Deepening Alpha-Beta Search with Transposition Tables** integrated alongside **Hierarchical Task Networks (HTN)** or **Monte Carlo Tree Search (MCTS)**. HTN delegates high-level macro objectives (e.g., "Build Army Base") to compress the branching factor b, allowing the low-level Alpha-Beta search to execute quickly within strict 50ms decision windows.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand